Team Project-Final
Team Deliverable 5 – Project Phase III
ITIS.3300 (Spring 2026)

Due Date: 4/26, 11:59pm
Submission:
- "Team Deliverable 5.pdf" Report (11pt, single-space, 5-7 pgs.): Canvas & GitHub
- Code: GitHub

Deliverables:
This is a finalization of the entire project. You will hand in a working program based on the specifications you previously developed for "Team Deliverable 4.pdf" along with new unit tests cases and system (functional) tests.

Hand-in and demo:
1. **Code:** All the source code for your working program: check them in the designated source code directory (**phase 3**) in your project repository.

2. **Report:** One document "Team Deliverable-5.pdf" that contains the following items: **NOTE**: The scope of "Team Deliverable 5.pdf" builds upon the content of "Team Deliverable 4.pdf"; therefore, all items in "Team Deliverable 5.pdf" should be updated to reflect these changes accordingly.
   a. Requirements

- List a set of requirements designated for this development phase.
- If the scope for this phase is unchanged, use the same requirements of phase 1 specified in Deliverable 2, but **more detailed descriptions** about the requirements should be provided.
  -
- If not, explain the reasons why the plan is changed and describe the updated plan including the modified set of requirements for all phases including the **detailed descriptions** of the requirements for phase 3.

Functional Requirements for Phase III:
FR1: Menu Browsing and Filtering
Description: The menu should display key item information such as the item name, price, description, and category. When a user selects a filter or enters a search term, the system should only show matching menu items. If no items match the search or filters, the system should provide a clear message instead of leaving the page blank.

FR2: Cart Management
Description: The cart should display each selected item, its quantity, item price, and calculated subtotal. Users should be able to increase or decrease quantities without restarting

the order. If an item is removed, the cart totals should update automatically. The system should calculate and display the subtotal, tax, delivery fee, service fee, tip, and final order total before the user submits the order.

FR3: Checkout Validation
Description: The checkout page should collect information such as order type, email address, cardholder name, card number, and delivery address when delivery is selected. The system should prevent users from submitting the checkout form if required fields are missing or invalid. Validation messages should clearly explain what information needs to be corrected. For pickup orders, delivery address fields should not be required unless delivery is selected

FR4: Order Submission and Storage
Description: When the user submits a valid order, the frontend should send the order data to the backend through the order API. The backend should receive the order information, generate or store an order ID, and save the order details in the SQLite database. Stored information may include customer email, order type, selected items, total cost, timestamp, and order status. The system should return a successful response when the order is saved correctly.

FR5: Confirmation
Description: The confirmation page should clearly show that the order was placed successfully. It should display the generated order ID so the user has a reference number for the order. The page may also display important order details such as order type, total cost, and selected items. If the order fails to submit, the user should not be sent to the confirmation page and should instead receive an error message.

FR6: Order History
Description: The order history page should retrieve stored order data from the backend and display previous orders in an organized format. Each order should include useful details such as order ID, date or time placed, order total, and selected items. This feature confirms that order data is being stored and can be retrieved later from the database.

Non-Functional Requirements for Phase III:
NFR1: Usability
Description: Users should be able to move through the main ordering process without confusion. Pages such as menu browsing, cart review, checkout, confirmation, and order history should be clearly labeled and easy to navigate. Buttons, forms, totals, and validation messages should be understandable to the user.

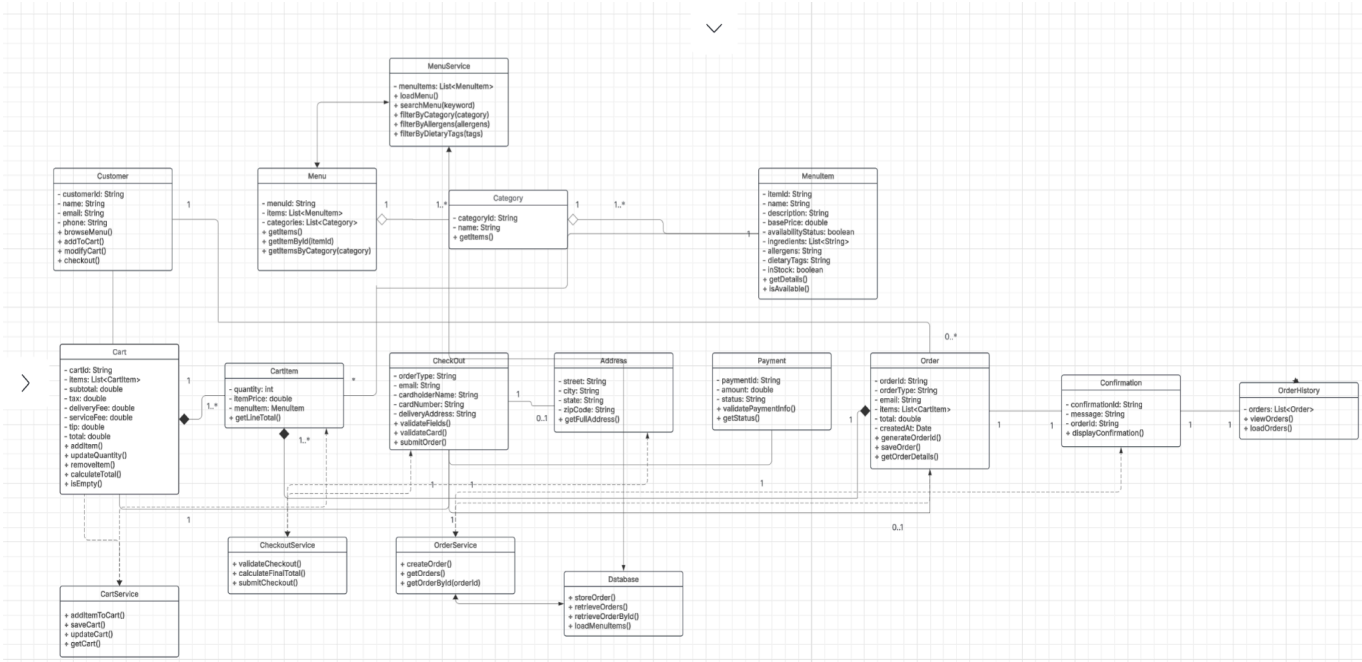
NFR2: Maintainability
Description: Frontend files should be separated by purpose, such as HTML for structure, CSS for styling, and JavaScript for interactivity. Backend files should organize routes, server logic, and database operations clearly. This modular structure makes the system easier to debug, test, and expand with additional features in the future.

NFR3: Reliability
Description: The system should correctly process valid orders, prevent incomplete checkout submissions, and return stored order history when requested. If invalid information is

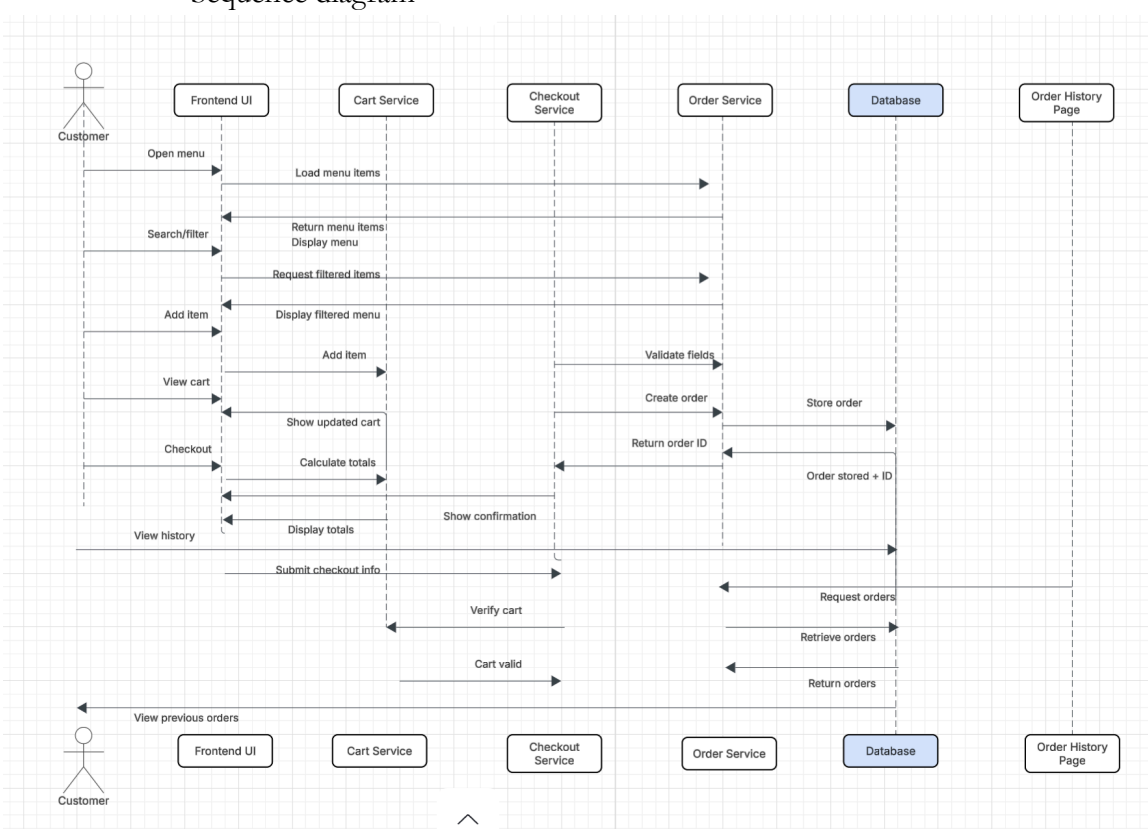
entered, the system should provide feedback instead of failing unexpectedly. The backend should handle order submission and retrieval consistently during normal use.

b. UML design. You must include the following diagrams:

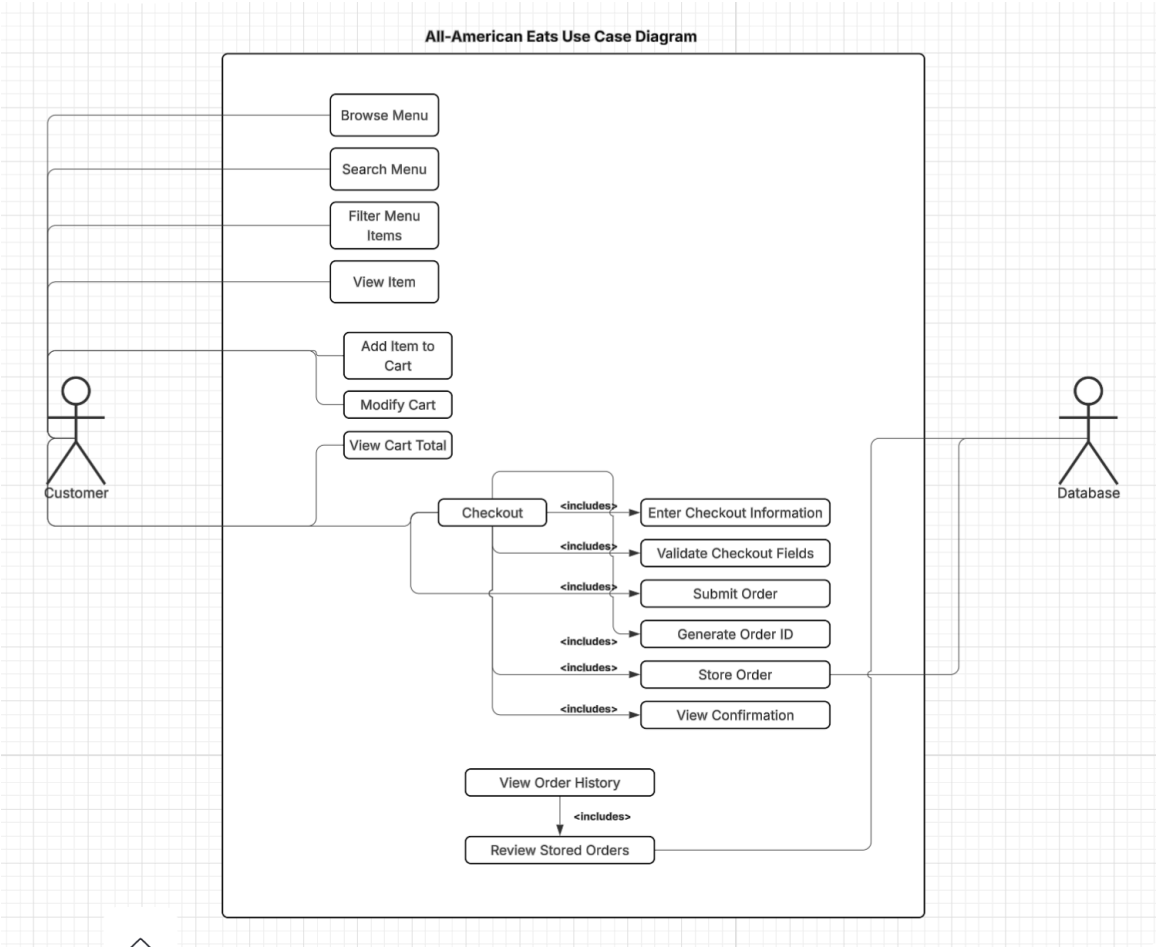
- Class diagram



- Sequence diagram



- Use case diagram



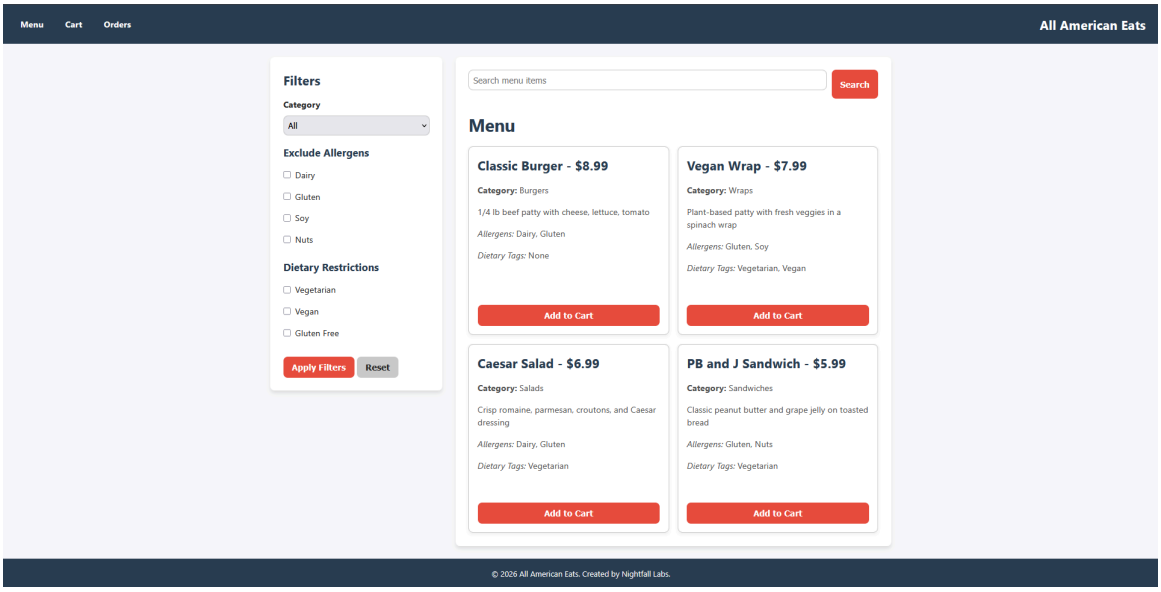
C. User Manual:

OVERVIEW:
Welcome to the All American Eats ordering application! This system allows users to browse our delicious menu, filter items based on their preferences, add selections to a cart, and place an order for delivery or pickup.

INSTALLATION (End User instructions):
For end-users, there is no installation required. Simply open your modern web browser (Google Chrome, Firefox, Edge) and navigate to the application URL provided by the restaurant (locally, this will be your `index.html` file).

HOW TO USE:

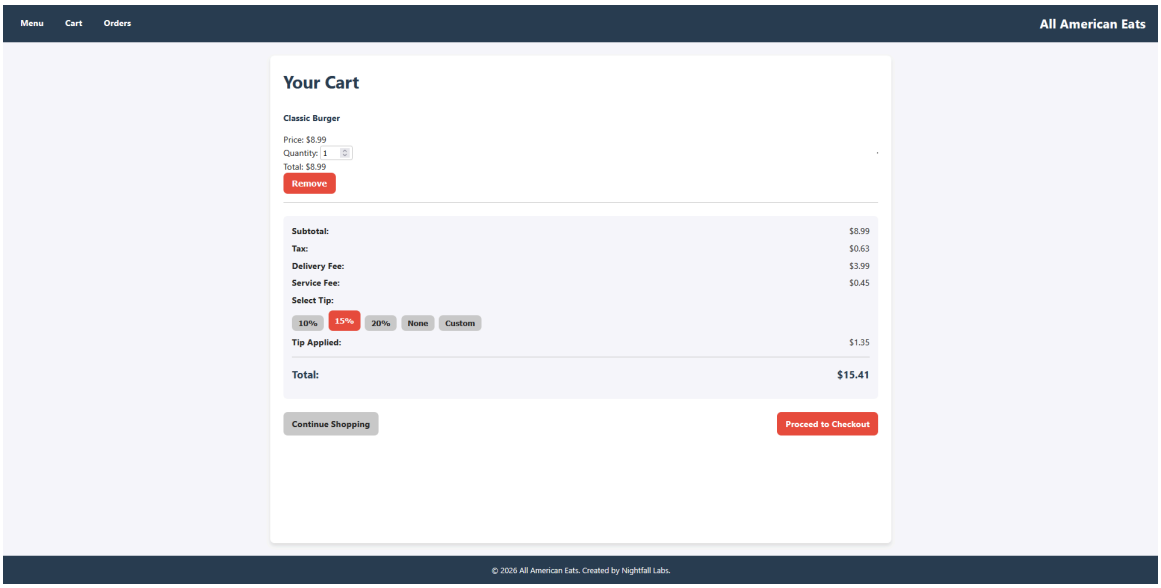
1. Browsing the Menu:



- Navigate to the Home page ('Menu' tab).
- Use the category filters on the left sidebar to show only specific types of items (such as Burgers, Wraps, or Salads).
- Use the dietary restrictions checkboxes or allergen exclusions to filter out unwanted items.
- Use the top search bar to look for a specific craving you have.

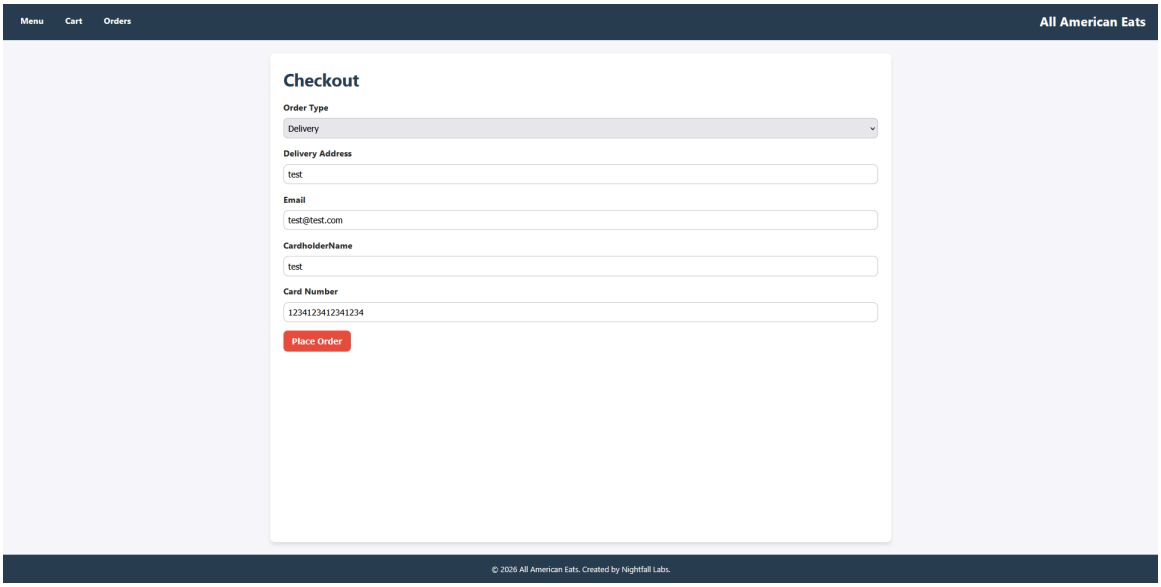
2. Adding to Cart:
   - Click on an item to view its details.
   - Add it to your cart (and select any customizations if applicable).

3. Viewing your Cart:



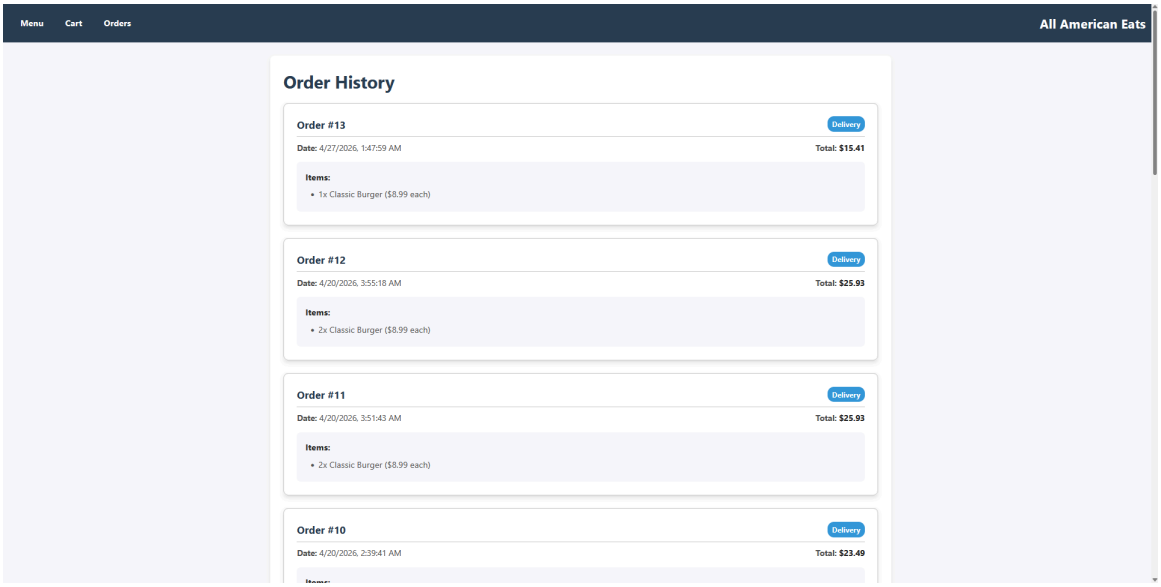
- Navigate to the 'Cart' page using the top navigation bar.
- Here you can review your items, subtotal, tax, fees, and adjust your tip.

4. Checkout:



- Proceed to checkout from the cart page.
- Select whether you want "Delivery" or "Pickup".
- Enter your email, cardholder name, and 16-digit card number.
- If "Delivery" is selected, enter your delivery address.
- Click "Place Order" to finalize your purchase!

5. Viewing Past Orders:



- Navigate to the 'Orders' page in the top menu to view your localized order history and totals.

D. INSTRUCTIONS TO COMPILE/RUN THE PROGRAM AND TEST CASES:

System Prerequisites:

- Node.js (version 14.x or higher) installed. This allows the backend server to run.

- NPM (Node Package Manager).

Compiling & running the Application:

1. Setting up the Backend:
   - Open a command line interface (Terminal/Command Prompt) in the root directory of the project.
   - Run the following command to download the server dependencies (reads from package.json): `npm install`
   - Change directories to the backend folder and start the application server: `cd src/backend` `node server.js`
   - You should see the message: "Connected to database" and "Server running on http://localhost:3000". Leave this terminal window open.
2. Running the Frontend:
   - Open the directory `src/frontend` in your file explorer.
   - Double-click on `index.html` to open the application in your default web browser.
   - Ensure the server is actively running in the background for database interactions (like adding cart items or retrieving orders) to succeed.

Running Test Cases:

- Currently, the application relies on manual acceptance testing to verify functionality.

- Open the application in your browser (`index.html`).
- Perform the following manual test scenarios to verify functionality:
- Test 1 (Menu filters): Check all dietary checkboxes and confirm only compliant menu items appear.
- Test 2 (Cart Validation): Attempt to proceed to checkout with an empty cart (the application should prevent this).
- Test 3 (Checkout form): Enter an invalid credit card number during checkout and verify that it requires exactly 16 digits.
- Test 4 (Order lifecycle): Complete a successful order, go to the "Orders" page, and ensure the new order ID correctly displays alongside the correct total.

E. Test Cases (unit tests + system (functional) tests)

List a set of test cases used for testing the working program including descriptions of tests (e.g., what functionality they test, and inputs/outputs for them). While unit testing focuses on the components that have been implemented in phase I and II, system (functional) testing should consider **the entire project** (components that have been implemented through all three phases).

Functionality/ Method	Input	Output	Pass/Fail

Menu Filtering	Select "Vegan" filter	Only vegan menu items displayed	Pass
Menu Search	Search "Burger" in search bar	Only burger-related items appear	Pass
Add to Cart	Click "Add to Cart" on item	Item appears in cart with correct quantity	Pass
Update Quantity	Increase item quantity to 2	Cart updates total and quantity correctly	Pass
Remove Item	Click remove button on item	Item is removed from cart	Pass
Empty Cart Checkout	Attempt checkout with empty cart	Error message prevents checkout	Pass
Checkout Validation	Leave email field blank	Error message shown	Pass
Invalid Card Input	Enter card number "1234"	Error: must be 16 digits	Pass
Delivery Option	Select delivery without address	Error message shown	Pass
Successful Order	Enter valid checkout info and submit	Confirmation page with order ID displayed	Pass
Order Storage	Complete an order	Order appears in order history page	Pass
Order History	Navigate to orders page	Previous orders displayed correctly	Pass

F. IMPLEMENTED FEATURES, LIMITATIONS, AND FUTURE PLANS

Successfully Implemented Features:

For Phase III, our team successfully finalized the main features of the All American Eats ordering system. The program allows users to browse the menu, search for items, use filters, add items to the cart, update item quantities, remove items, and review the cart total before checkout. The checkout page also collects the needed customer and payment information and validates required fields before the order can be submitted. When an order is completed successfully, the system displays a confirmation page with an order ID. The order history

feature also allows previous orders to be viewed, which shows that the order information can be stored and retrieved.

Limitations:
One limitation is that the system is still a class project version of a restaurant ordering system, so some features are simplified. For example, payment processing is not connected to a real payment provider, and some order information is used mainly for demonstration and testing purposes. The menu items and filters are also limited to the sample data included in the project. Another limitation is that testing is mostly based on manual functional tests instead of a larger set of automated unit tests. Because of this, future changes would need more formal automated testing to make sure the system stays reliable.

Feedback and Improvements:
During review, the team noticed that the system worked overall, but some parts could be made more consistent. One improvement would be making sure all cart, checkout, fee, tax, and tip calculations are handled in the same way across the program. Another improvement would be making the user interface cleaner and more polished so the ordering process feels smoother.

Future Plans:
If we continued working on this project next semester, we would add more realistic restaurant features such as user accounts, saved payment methods, real payment integration, better order tracking, and more advanced menu customization. We would also add more automated unit tests and system tests to reduce bugs and make the project easier to maintain.

G. REFLECTION
For this final phase, our team focused on bringing the project together into a complete working version and making sure the report matched the final system. We were able to organize the main restaurant ordering features, including menu browsing, filtering, cart management, checkout validation, order confirmation, and order history. Even though the code did not significantly change during this phase, we still reviewed the project as a whole and updated the documentation to better explain what the system does and how it should be tested.

One thing that went well was that our project already had the main structure and features in place from the earlier phases, so this phase was mostly about finalizing and making the deliverable clearer. The requirements, test cases, user manual, and reflection helped show how the different parts of the system connect together.

The project could be improved by polishing the user interface, making calculations more consistent across pages, and adding more realistic restaurant features such as real payment processing, user accounts, and better order tracking. Overall, this phase helped us finalize the project and better understand what would need to be improved if the project continued in the future.

H. Member contribution table (should describe who wrote what components or classes of the system and what parts of the report). Add more rows as needed.

Member name	Contribution description	Overall Contribution (%)	Note (if applicable)
Thomas Clayton	-Created document & shared will all group members -Part A & E	25%	
Danny Moncada	Parts F & G of Report	25%	
Alex Lopez Robles	Parts C & D of the Report	25%	
Jayant Thathireddy	Added deliverable, meeting minutes, and note to Github	5%	
Isaac Baraki	Created UML class, sequence, and user diagrams	20%	

3. The updated meeting minutes (project repository): don't include this in the report.

Document formats: Your documents may be in any of the following formats (1) MS Word (2) Adobe Acrobat (pdf).

Grading: You will be graded on how well you have completed each of the required elements and on your adherence to the directions given. Credit will be given for the completeness and level of detail placed into each of the required elements.

Team Policy: Each team member will receive the same grade for their assignment, however, if particular members are not participating, adjustments to that individual's grade may be made.

What to Hand In: At 11:59pm on the due date the instructor or TAs will take a snapshot of each of your directories and use this for grading. Create a "**Note-deliverable-5**" file (a top level, plain text) to indicate the names of deliverable files and their locations for this deliverable.